

How Prod AI runs in production.

Five planes with one job each. The control plane decides, sandboxes run untrusted code, the runtime serves live apps, and every model call and every agent action passes a gateway that can price it, cap it or ask a person first. The prototype you can click today implements the product surface of this design; the table at the end says exactly which parts are real.

- Diagram (PNG)
- Diagram + notes (PDF)
- Full write-up

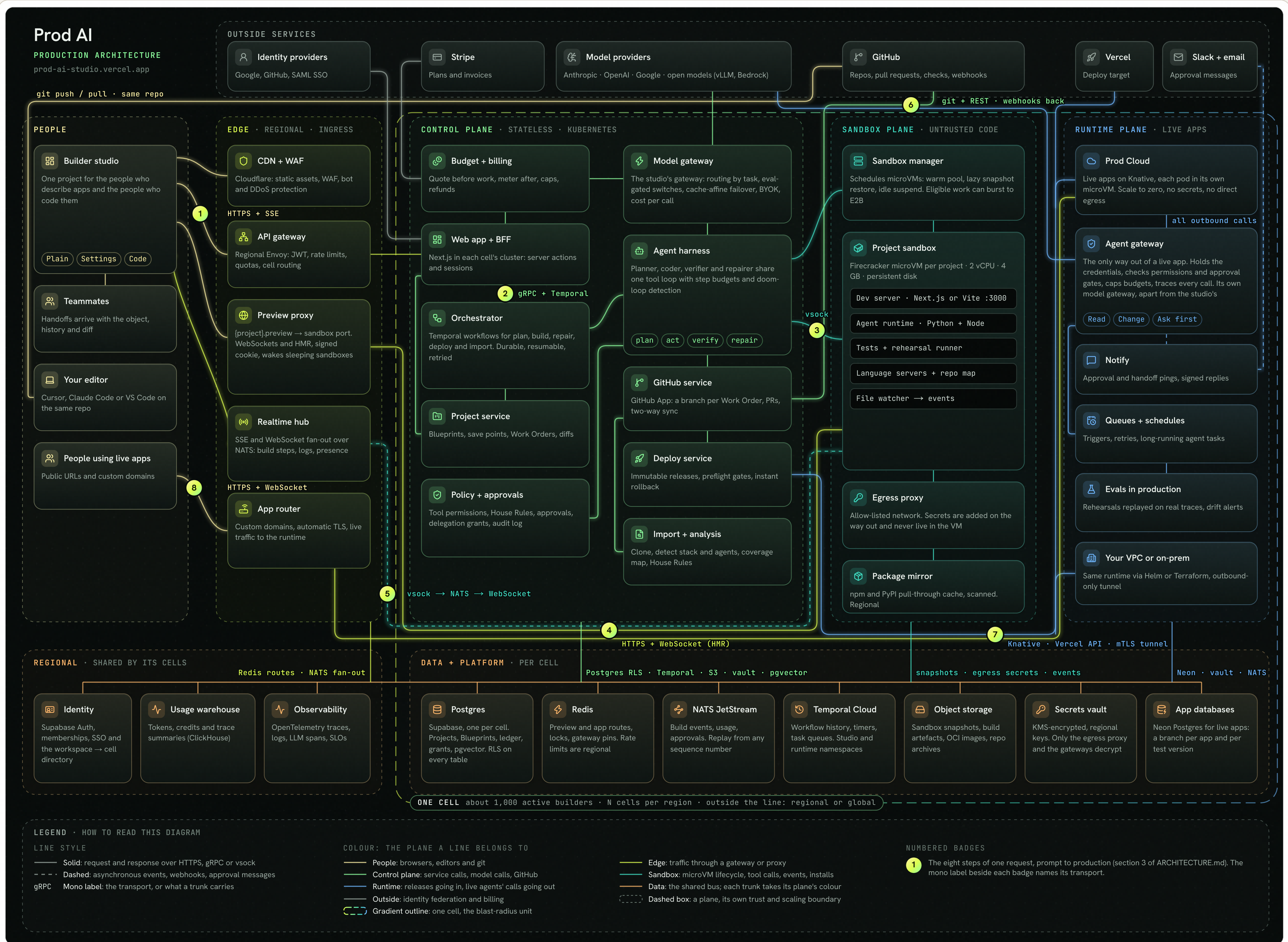


Fig. 1 Five planes, one job each: control, sandboxes, runtime, data and the gateways every model call and agent action passes. Numbers mark the eight flows below.

The eight flows on the diagram

- 1 Describe

The studio sends the brief through the API gateway. Nothing runs yet: the planner returns a Blueprint and a priced Work Order.
- 2 Plan

The orchestrator starts a durable workflow. The harness asks the model gateway, which picks a model per task and meters every call.
- 3 Build in a sandbox

Approved work runs inside the project's microVM: write files, install, run, test, rehearse agents. Every step is checkpointed.
- 4 Live preview

The preview proxy maps the project's subdomain to the sandbox dev server, including WebSockets for hot reload.
- 5 Show the work

File, test and step events stream back through the realtime hub, so people see progress in plain English, not a spinner.
- 6 Branch + PR

Each Work Order becomes a branch and a pull request through the GitHub App. Engineers' pushes sync back into the Blueprint or become code they own.
- 7 Ship

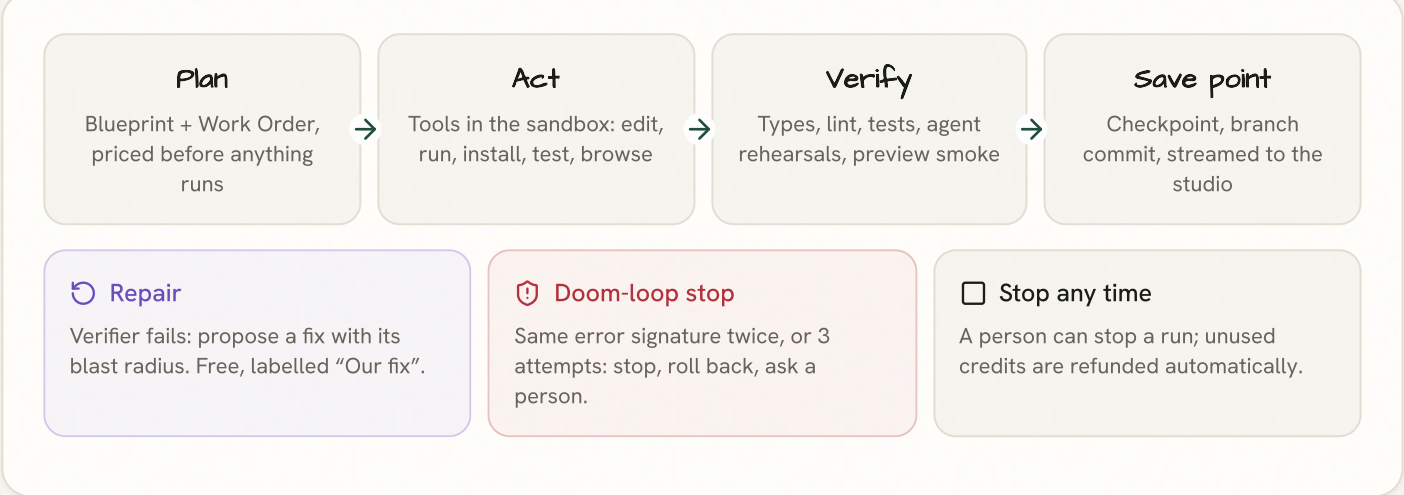
Preflight passes, the deploy service builds one immutable release and rolls it out to Prod Cloud, Vercel or your VPC.
- 8 Governed agents

The agent gateway is every live app's only way out: credentials, approval gates, caps and traces. Model calls use the runtime's own model gateway.

THE AGENT HARNESS

Plan, act, verify, repair. With a budget and a way out.

The model decides; deterministic code does the rest. The planner writes a Blueprint (structured JSON), code generation turns it into files, and a tool loop edits, runs and tests inside the sandbox. A verifier gates every step. When a check fails, the repairer tries a fix, but only within a step and cost budget. The same error twice means a doom loop: it stops, rolls back to the last save point and hands the problem to a person with the full context. Our own fixes are never billed.



SANDBOXING

One microVM per project, because the code is untrusted.

OPTION	VERDICT	ISOLATION	START	FIT FOR AGENTIC APPS
Firecracker microVMs	Chosen	Own kernel per project	~150 ms from snapshot	Runs Node and Python agents, real ports, persistent disk
gVisor containers	Fallback	User-space kernel	~1 s	Good isolation, some syscall and I/O overhead
Plain Docker	Rejected	Shared host kernel	~1 s	One escape reaches every tenant
Browser WebContainers	Rejected	Browser tab	Instant	Node only: no Python agents, no long-running jobs

SCALE

Thousands of people at once, without thousands of idle machines.

5,000 people in the studio at once	~1,500 sandboxes awake (30% active, the rest snapshotted)	~150 ms resume from a warm snapshot	≤ 3 repair attempts before a person is asked
---------------------------------------	--	--	---

- ✓ The control plane is stateless; long work lives in durable workflows, so a pod restart never loses a build.
- ✓ The model gateway queues by priority per provider and key, fails over between providers, and caches prompts.
- ✓ Sandboxes suspend to a snapshot after 10 idle minutes and resume in about 150 ms from a warm pool.
- ✓ Events fan out over NATS; Postgres uses row-level security, read replicas and partitioned event tables.

HONEST ABOUT THE SEAMS

What the prototype runs today, and what production adds.

Real runs in the live prototype. **Stand-in** is simulated or sandboxed, and labelled in the product. **Designed** is specified in ARCHITECTURE.md and not built yet.

PART	IN THE LIVE PROTOTYPE	IN PRODUCTION
Studio, auth, data Real	Next.js 16 on Vercel, Supabase Auth (Google, email, guest sessions you can keep), Postgres with row-level security on every table	The studio backend runs in each regional cell (service identities, residency, long-lived streams); Vercel keeps the marketing site. Plus SAML SSO, SCIM and regional data residency
Planner Real	Claude plans a structured Blueprint, streamed live; code expands it deterministically and validates every reference	Same contract, routed through the model gateway
Change requests Real	Claude returns typed edits; code resolves names and emits validated operations, with one self-repair retry; each change is a priced Work Order and a save point	Same, executed by the harness in a sandbox; parallel Work Orders rebase onto a versioned Blueprint
Model gateway Model switch by environment: real Routing gateway: designed	One getModel() seam that picks the model from the environment, with per-call token and cost metering and daily budgets per person. One provider (Anthropic) and no gateway service	A gateway service: routing by task, eval-gated switches, failover in the middle of a tool loop, prompt caching, BYOK, and a separate deployment for live apps
Approvals and the agent gateway Approval gate in the playground: real Production gateway: designed	Claude + AI SDK tool approval: tools marked "Ask first" pause for a person before running. Tools run on sandboxed sample data; nothing holds credentials or controls egress yet	Every production tool call passes an egress gateway that holds the credentials, enforces approvals and caps, tracks untrusted content per run, and traces the call
Build + repair Build: simulated, labelled Repair decision: real	Deterministic build timeline with a real repair decision that changes the Blueprint and creates a save point	Full tool loop inside microVMs with verifier and budgets
Sandbox + preview Simulated, labelled	Preview renders the Blueprint with a spec renderer inside the studio; no generated or imported code runs	Firecracker microVM per project behind the preview proxy; imported repos run as themselves, with a preview per exposed port
GitHub Public repo analysis: real Pushes, PRs, sync: sandboxed	Reads any public repo, detects stack and agent frameworks, maps its agents, writes House Rules; pushes and PRs are sandboxed	GitHub App with a branch and worktree per Work Order, a merge queue and two-way sync
Deploy Public/live URL + rollback: real Builds and releases: designed	A public /live/... URL serves a published Blueprint snapshot through the spec renderer, with rollback. No build runs; Vercel and VPC targets are sandboxed	Immutable releases, canary rollout, instant rollback